

Earth in Hues

Sandesh Bhandari (github.com/snes19xx)

March 15, 2026

Contents

Earth in Hues	1
1. The grid	2
2. Area weighting	2
3. Surface categories	2
4. Averaging colour in the wrong space	3
5. Does one colour describe a category?	4
6. What Earth looks like from space	4
7. Twenty-four years, two satellites	5
8. Outputs	6
Limitations	7

Earth in Hues

What colour is the Earth?

This notebook computes the area-weighted mean colour of every surface category on the planet, month by month, from satellite imagery. It is the methods document for the write-up.

This project answers three questions, in order:

1. What colour is each part of Earth's surface?
2. What colour would it be seen from space, with the atmosphere and clouds in the way?
3. Has any of it changed over twenty-four years?

Every function used here is in `src/earthhues/`.

```
import json
import os
from pathlib import Path

import numpy as np

# paths below are relative to the repository root
root = Path.cwd()
while not (root / "pyproject.toml").exists() and root != root.parent:
    root = root.parent
os.chdir(root)

from earthhues.sources import Sources, MONTHS
from earthhues.masks import build_masks, SURFACE_CATEGORIES, CATEGORIES
from earthhues.weights import cosine_weights, latitudes

sources = Sources("DATA", "cache")
grid = sources.grid
print("grid   ", grid.shape, grid.crs)
print("extent ", grid.transform * (0, 0), "to", grid.transform * (grid.width, grid.height))

grid    (1800, 3600) EPSG:4326
extent  (-180.0, 90.0) to (180.0, -90.0)
```

1. The grid

Everything is aligned to the January Blue Marble composite: a 1800 x 3600 grid at 0.1 degree spacing in EPSG:4326. Elevation and land cover are resampled onto it before anything is computed.

Input	Resampling	Why
GEBCO elevation	average	continuous field, preserves the mean
MODIS land cover	nearest neighbour	class integers, interpolation would invent classes

```
dem = sources.dem
landcover = sources.landcover
print("elevation ", dem.shape, dem.dtype, f"{dem.min():.0f} to {dem.max():.0f} m")
print("land cover ", landcover.shape, landcover.dtype, f"{landcover.min():.0f} to {landcover.max():.0f}")

elevation (1800, 3600) float32 -10441 to 6395 m
land cover (1800, 3600) uint8 0 to 16
```

2. Area weighting

In an equirectangular grid every pixel covers the same angular area, but not the same physical area. Longitude lines converge at the poles, so a pixel at 60 degrees north covers half the ground a pixel at the equator does.

Model Earth as a sphere of radius R . The circle of latitude at ϕ has circumference $2\pi R \cos \phi$, so a longitude step $\Delta\lambda$ spans

$$\Delta x(\phi) = R \cos \phi \Delta\lambda$$

and the pixel area is

$$A(\phi) = R^2 \cos \phi \Delta\phi \Delta\lambda.$$

$R^2 \Delta\phi \Delta\lambda$ is constant across the grid, so the relative weight of a pixel reduces to $w(\phi) = \cos \phi$. Skip this and the poles are counted many times over.

```
weights = sources.weights
lats = latitudes(grid)

for label, lat in [("equator", 0), ("New York", 40.7), ("Oslo", 59.9), ("pole", 89.95)]:
    row = int(np.argmin(np.abs(lats - lat)))
    print(f"{label:9s} {lats[row]:+7.2f} deg weight {weights[row, 0]:.3f}")

equator +0.05 deg weight 1.000
New York +40.75 deg weight 0.758
Oslo +59.95 deg weight 0.501
pole +89.95 deg weight 0.001
```

3. Surface categories

Fourteen categories. Ten of them come straight from the IGBP class of each pixel, split by the sign of the elevation so ocean and inland water separate. Four are aggregates.

The ten surface categories partition the grid exactly. Every pixel belongs to one, no pixel belongs to two.

```
mask = build_masks(dem, landcover)
stack = np.stack([mask[c] for c in SURFACE_CATEGORIES])
counts = stack.sum(axis=0)

print("pixels in exactly one category:", int((counts == 1).sum()))
print("pixels in none:", int((counts == 0).sum()))
print("pixels in more than one:", int((counts > 1).sum()))

total = weights.sum()
```

```

print()
for name in SURFACE_CATEGORIES:
    print(f" {name:14s} {100 * weights[masks[name]].sum() / total:6.3f} %")
print(f" {'ocean total':14s} {100 * weights[masks['Total Ocean Mean']].sum() / total:6.3f} % (accepted 70.8)

pixels in exactly one category: 6480000
pixels in none: 0
pixels in more than one: 0

Oceans      70.736 %
Fresh Water  0.866 %
Snow and Ice 2.883 %
Deserts     4.033 %
Forests     4.396 %
Grasslands  11.562 %
Shrublands  2.745 %
Croplands   2.445 %
Wetlands    0.216 %
Urban Areas 0.118 %
ocean total 70.877 % (accepted 70.8)

```

4. Averaging colour in the wrong space

The GeoTIFF stores gamma-encoded sRGB, not light. The stored value c relates to linear reflectance L by

$$L = \left(\frac{c + 0.055}{1.055} \right)^{2.4}$$

above the small linear toe. Averaging the stored values answers the wrong question, because the encoding is concave: by Jensen's inequality, the mean of the encoded values is always below the encoded mean.

The distinction matters when you average. Take a simple example: a picture that is half black and half white. If you average the pixel values directly, you get $(0 + 255)/2 = 128$, or mid-grey (#808080). But if you first convert the values back to actual light, average them, and then convert the result back, you get 188 (#bcbcbc). That second result is the correct one: half the light really is half the light, and half the light looks much brighter than half the code value.

The result is too dark, and the size of the error grows with how much contrast the category contains.

The fix is to decode, average, then re-encode.

```

from earthhues.color import weighted_mean_srgb, weighted_mean_linear, rgb_to_hex
from earthhues.colorspace import srgb_to_lab, delta_e_2000
from earthhues.extract import read_month

rgb, valid = read_month(sources, "jan")

print(f"{'category':18s} {'sRGB mean':>10s} {'linear mean':>12s} {'dE00':>6s}")
for name in ("Oceans", "Snow and Ice", "Deserts", "Forests", "Total Earth Mean"):
    mask = masks[name] & valid
    w = weights[mask]
    samples = np.stack([rgb[i][mask] for i in range(3)], axis=1)

    gamma = weighted_mean_srgb(samples, w)
    correct = weighted_mean_linear(samples, w)
    shift = delta_e_2000(srgb_to_lab(gamma / 255), srgb_to_lab(correct / 255))
    print(f"{'category':18s} {rgb_to_hex(*gamma):>10s} {rgb_to_hex(*correct):>12s} {shift:6.1f}")

```

Uniform categories barely move. Ocean shifts 0.6, snow 0.3. Whole Earth shifts 15.2, from a near-black #28292d to a mid grey #565552.

That pattern is the prediction. The error should scale with the spread inside a category, so measure the spread and check.

```

methods = json.load(open("data/earth_hues_methods.json"))
stats = json.load(open("data/earth_hues_stats.json"))

```

```

pairs = [
    (s["categories"][n]["dispersion"], m["categories"][n]["gamma_shift"])
    for s, m in zip(stats, methods)
    for n in s["categories"]
    if s["categories"][n].get("pixels")
]
spread, shift = np.array(pairs).T
print(f"{len(pairs)} category-months")
print(f"correlation between within-category spread and the gamma error: r = {np.corrcoef(spread, shift)[0, 1]}")

168 category-months
correlation between within-category spread and the gamma error: r = 0.940

r = 0.940 across 168 category-months. The error is not a constant offset. It is governed by variance, which is what the concavity argument predicts.

```

5. Does one colour describe a category?

The mean is one number standing in for millions of pixels. For some categories that is honest and for others it is not, so report the spread alongside it.

```

january = stats[0]["categories"]
rows = sorted(
    ((n, e) for n, e in january.items() if e.get("pixels")),
    key=lambda kv: kv[1]["dispersion"],
)
print(f"{'category':18s} {'mean':>9s} {'L* p05':>7s} {'p50':>6s} {'p95':>6s} {'spread':>7s}")
for name, entry in rows:
    light = entry["lightness"]
    print(f"{'name':18s} {'entry['mean']:>9s} {light['p05']:.7f} {light['p50']:.6f} {light['p95']:.6f} {entry[

```

category	mean	L* p05	p50	p95	spread
Oceans	#060a16	1.3	1.6	1.7	1.4
Total Ocean Mean	#070b17	1.3	1.6	1.8	1.7
Snow and Ice	#ebeef0	85.8	94.5	99.0	3.2
Deserts	#ae9675	42.1	62.3	81.0	9.5
Urban Areas	#575340	15.1	27.8	63.3	11.7
Forests	#4f5650	10.2	16.5	70.8	19.5
Croplands	#7b776c	19.6	33.0	91.1	20.6
Wetlands	#adb4b5	13.8	81.0	93.8	23.7
Total Earth Mean	#565552	1.6	1.6	90.2	24.5
Mountains	#96928a	14.3	44.9	95.1	24.7
Grasslands	#878780	16.6	35.4	94.1	25.5
Shrublands	#a9a29c	31.9	46.5	96.4	26.2
Total Land Mean	#98948c	13.8	44.5	95.3	27.4
Fresh Water	#a6a9a9	1.4	52.8	95.5	31.2

Only ocean and snow are well described by a single colour. Fresh Water is the worst: frozen lakes and open tropical water share one class and nothing sensible sits between them.

6. What Earth looks like from space

Blue Marble is atmospherically corrected and cloud cleared. It records what the ground reflects, which is not what a distant observer sees. Two layers sit in between.

Rayleigh scattering. Optical depth goes as λ^{-4} , so blue scatters about four times more strongly than red, and that light is added along every viewing path. Optical depth is thinned by terrain height through the barometric scale height, and the solar zenith angle comes from latitude and the day of year.

Cloud. About two thirds of the planet at any moment.

```

from earthhues.atmosphere import BAND_WAVELENGTHS_UM, rayleigh_optical_depth

tau = rayleigh_optical_depth(BAND_WAVELENGTHS_UM)

```

```

for band, wavelength, t in zip("RGB", BAND_WAVELENGTHS_UM, tau):
    print(f" {band} {wavelength * 1000:.0f} nm tau = {t:.4f}")
print(f"\nblue scatters {tau[2] / tau[0]:.1f} times more strongly than red")

R 645 nm tau = 0.0509
G 555 nm tau = 0.0938
B 469 nm tau = 0.1867

```

blue scatters 3.7 times more strongly than red

```

space = json.load(open("data/earth_hues_space.json"))
views = space["records"][0]["views"]

```

```

print(f"{'category':18s} {'surface':>9s} {'+atmosphere':>12s} {'+clouds':>9s}")
for name in ("Oceans", "Forests", "Deserts", "Snow and Ice", "Total Earth Mean"):
    print(f"{'name':18s} {'views['surface'][name]['hex']:>9s} "
          f"{'views['atmosphere'][name]['hex']:>12s} {'views['space'][name]['hex']:>9s}")

```

category	surface	+atmosphere	+clouds
Oceans	#060a16	#30415c	#aaabae
Forests	#4f5650	#57626b	#aeafb1
Deserts	#ae9675	#ad9781	#b4a598
Snow and Ice	#ebeef0	#e7e7e8	#dbdbdc
Total Earth Mean	#565552	#5e6370	#adadaf

The ocean goes from #060a16 to #30415c while the Sahara barely moves. Earth reads as a blue planet because most of it is dark enough for scattered blue light to win, not because seawater is blue.

Checking the model against a camera The cloud layer has one fitted parameter, an effective optical thickness set so that whole-Earth albedo matches the observed value. Fitting a brightness says nothing about colour balance, so the blue-to-red ratio is a free prediction. DSCOVR EPIC photographs the full sunlit disk from a million miles away and can be compared directly.

```

from earthhues.color import hex_to_rgb
from earthhues.colorspace import srgb_to_linear

```

```

epic = json.load(open("data/earth_hues_epic.json"))

```

```

def blue_over_red(hex_value):
    linear = srgb_to_linear(np.array(hex_to_rgb(hex_value)) / 255)
    return linear[2] / linear[0]

```

```

for label, view in (("surface", "surface"), ("+ atmosphere", "atmosphere"), ("+ clouds", "space")):
    print(f" model {label:14s} {blue_over_red(views[view]['Total Earth Mean']['hex']):.3f}")

```

```

observed = np.array(epic["mean"]["reflectance"])
print(f" EPIC observed {observed[2] / observed[0]:.3f} ({len(epic['frames'])} frames)")

model surface      0.907
model + atmosphere  1.448
model + clouds     1.026
EPIC observed      1.041 (11 frames)

```

The bare surface leans red at 0.907. Atmosphere alone overshoots to 1.448. Add cloud and the model lands on 1.026 against 1.041 measured by the camera, agreeing to within 0.003 in every channel on a quantity that was never fitted.

7. Twenty-four years, two satellites

One climatological year cannot say whether anything is changing. For that the same measurement runs over 276 eight-day MODIS composites from 2001 to 2024, six per year, from Terra and Aqua.

The second satellite is the point. Terra and Aqua observe the same planet and are calibrated independently, so a change that is real appears on both. A change that appears on one is the instrument. Deserts are radiometrically stable and set the noise floor.

```
trends = json.load(open("data/earth_hues_trends.json"))
agreement = trends["agreement"]
adjusted = trends["control_adjusted"]
floor = abs(trends["trends"]["terra"]["Deserts"]["slope_per_decade"])

print(f"desert noise floor: {floor:.3f} L* per decade\n")
print(f"{'category':18s} {'terra':>8s} {'aqua':>8s} {'gap':>8s} verdict")
for name, entry in sorted(agreement.items(), key=lambda kv: -abs(kv[1]["difference"])):
    clears = adjusted["terra"][name]["above_floor"] and adjusted["aqua"][name]["above_floor"]
    verdict = "real trend" if clears else ("sensors disagree" if entry["exceeds_control"] else "")
    print(f"{name:18s} {entry['terra']:>8.3f} {entry['aqua']:>8.3f} {entry['difference']:>8.3f} {verdict}")

desert noise floor: 0.200 L* per decade
```

category	terra	aqua	gap	verdict
Oceans	+0.404	-0.234	+0.639	sensors disagree
Total Ocean Mean	+0.400	-0.229	+0.629	sensors disagree
Urban Areas	-0.541	-0.887	+0.346	real trend
Total Earth Mean	+0.064	-0.202	+0.266	
Fresh Water	+0.349	+0.137	+0.212	
Croplands	-0.407	-0.608	+0.201	
Grasslands	-0.119	-0.241	+0.123	
Forests	-0.486	-0.556	+0.070	real trend
Snow and Ice	+0.103	+0.165	-0.062	
Mountains	-0.385	-0.424	+0.039	
Deserts	-0.200	-0.236	+0.036	
Shrublands	-0.124	-0.156	+0.032	
Wetlands	-0.074	-0.047	-0.028	
Total Land Mean	-0.192	-0.200	+0.008	

Read the ocean row first. Terra reports significant brightening at +0.404 L* per decade. Aqua reports -\$0.234 and no significance. Their disagreement is eighteen times what separates the two satellites over deserts. Terra is known to drift on dark targets, and a project built on one satellite would have published ocean brightening as a finding.

Every land category falls inside the band, most with the two satellites nearly on top of each other. Total Land differs by 0.008 L* per decade between two independently calibrated instruments. That agreement is what makes the land numbers usable.

Two categories clear the desert floor on both satellites:

- **Forests**, darkening at -\$0.49 and -\$0.56 L* per decade. Consistent with denser canopy absorbing more visible light, the optical signature of the greening reported from vegetation indices.
- **Urban Areas**, at -\$0.54 and -\$0.89.

Total Land is statistically significant on both satellites and still not claimable. Its trend sits within 0.01 of the desert control, so it cannot be separated from drift they share. Statistical significance and physical meaning are not the same test.

8. Outputs

File	Contents
data/earth_hues.json	monthly mean colour per category, linear light
data/earth_hues_methods.json	five estimators side by side, with the gamma error spread, percentiles and area share
data/earth_hues_stats.json	surface, atmosphere and cloud views
data/earth_hues_space.json	observed disk colour from DSCOVR
data/earth_hues_trends.json	annual means, trends and the satellite cross-check
data/raster/categories.png	category index per pixel, for the map

Regenerate any of them with `make colors`, `make stats`, `make space`, `make epic`, `make masks`, `make fetch` then `make timeseries`.

Limitations

- Blue Marble is a display product with NASA's own tonal stretch. Treating it as sRGB is the standard assumption and it is still an assumption.
- Cloud fraction counts an optically thin pixel as fully overcast, so the effective optical thickness is a fitted parameter, not a measured one.
- The modelled seasonal cycle correlates with EPIC at $r = 0.644$ but reproduces about an eighth of the observed amplitude. Three cloud days per month is thin sampling.
- Rayleigh scattering is single scattering only, which underestimates the blue path reflectance by roughly ten percent.
- Mountains overlap other categories by construction, so they are excluded from the partition used for the map.